Pràctica OpenMP
Computació d'Altes Prestacions

Miguel Arlindo Correia Salgueiro - 1747746
Oriol Figueras Ollé - 1748144

Índex

1 Introducció **3**
1.1 Compilació i execució 3

2 Estratègia de paral·lelització **5**
2.1 Paral·lelització de *feed_input()* 5
2.2 Paral·lelització de *init_deltas()* 6
2.3 Paral·lelització de *forward_prop()* 9
2.4 Paral·lelització de *compute_loss()* 11
2.5 Paral·lelització de *back_prop()* 13
2.6 Paral·lelització de *update_weights()* 16

3 Anàlisi de rendiment **18**
3.1 Fine-tuning de *forward_prop()* 18
3.2 Fine-tuning de *back_prop()* 23
3.3 Fine-tuning global 31
3.4 Comparació amb les dades de referència 35

4 Problemes trobats **36**
4.1 Concepte de SIMD 36
4.2 Baixada d'encerts sense la directiva `#pragma omp single` (Condició de carrera) 36
4.3 Augment del temps d'execució a causa de l'ús de la directiva `#pragma omp atomic` 36

5 Conclusions **38**

6 Ús de IA **39**

1 Introducció

L'objectiu principal d'aquesta pràctica consisteix en paral·lelitzar una implementació escrita en *C* d'una xarxa neuronal de tipus perceptró fent servir una aproximació de paral·lelització multi-core. S'usa OpenMP per fer l'implementació.

S'usa també Git com a eina de control de versions.

1.1 Compilació i execució

Per a compilar aquest projecte s'ha creat un *makefile* per fer la compilació més senzilla. En el *makefile* hi ha definides dues maneres de compilar, una amb la paral·lelització implementada i l'altra de manera seqüencial. A continuació es mostra el contingut del fitxer:

```
CC     := gcc
OPT    := -Ofast
LDLIBS := -lm

SRC := main.c \
       common/common.c \
       configuration/config.c \
       layer/layer.c \
       randomizer/randomizer.c \
       initialize/initialize.c \
       training/training.c

HDR := $(wildcard *.h */*.h)

.PHONY: all seq par clean

all: seq par
seq: exec_seq
par: exec_par

exec_seq: $(SRC) $(HDR)
        $(CC) $(OPT) $(SRC) -o $@ $(LDLIBS)

exec_par: $(SRC) $(HDR)
        $(CC) $(OPT) -fopenmp $(SRC) -o $@ $(LDLIBS)

clean:
        rm -f exec_seq exec_par
```

D'aquesta manera, amb un simple `make seq` o `make par` ja es crea l'executable

corresponent, amb els noms de `exec_seq` i `exec_par` respectivament. També es pot usar `make all` per generar tots dos executables. Per executar el programa simplement cal fer `./exec_seq` o `./exec_par`.

Es poden eliminar els executables també usant el *makefile*, amb `make clean`.

Per a fer útil el *makefile* creat, s'ha modificat el *seq.sub* donat i s'ha creat un *par.sub* addicional.

El fitxer *seq.sub* és el següent:

```
#!/bin/bash
#SBATCH -N 1 # number of nodes
#SBATCH --exclusive
#SBATCH --partition=nodo.q

module add gcc/13.2.1

make seq

## perf stat ./exec_seq configuration/configfile.txt
./exec_seq configuration/configfile.txt
```

I el fitxer *par.sub* està construït de manera similar, amb la lògica addicional de poder passar el nombre de threads com a argument del *sbatch*:

```
#!/bin/bash
#SBATCH -N 1 # number of nodes
#SBATCH --exclusive
#SBATCH --partition=nodo.q

THREADS=${1:-4} # default number of threads is 4

module add gcc/13.2.1

make par

export OMP_NUM_THREADS=$THREADS
echo "Threads: $OMP_NUM_THREADS"

## perf stat ./exec_par configuration/configfile.txt
./exec_par configuration/configfile.txt
```

Cal destacar que les ordres exactes per tal de enviar les execucions, són:

```
# Seqüencial
sbatch seq.sub

# Paral·lel amb n threads
sbatch par.sub n
```

2 Estratègia de paral·lelització

L'estratègia de paral·lelització usada en aquesta pràctica és una paral·lelització basada en bucles. S'ha escollit aquesta estratègia per la quantitat de *for* que hi ha en el *training.c* i per la dificultat afegida de la paral·lelització basada en tasques, que implicaria una modificació substancial del codi base.

2.1 Paral·lelització de *feed_input()*

Aquesta funció es pot paral·lelitzar, ja que les operacions dins el bucle de la funció són independents les unes de les altre. Tot i això, pot no suposar un gran avantatge, perquè cada crida d'aquesta funció només requereix fer una copia de 1024 valors del patró d'entrada i la sobrecàrrega de crear i sincronitzar threads pot ser superior al temps estalviat al paral·lelitzar.

La funció paral·lelitzada és:

```
1 void feed_input(int i) {
2     #pragma omp parallel for
3     for (int j = 0; j < num_neurons[0]; j++)
4         lay[0].actv[j] = input[i][j];
5 }
```

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *feed_input()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,5441 s	41,4919 s	42,0177 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
41,8789 s	43,8571 s	44,0709 s

Taula 1: Taula comparativa de temps de programa amb *feed_input()* paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,5441 s	42,0085 s	42,3634 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
44,7099 s	44,0000 s	42,2120 s

Taula 2: Taula comparativa de temps de programa amb *feed_input()* paral·lelitzat amb 8 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,5441 s	42,9662 s	42,7769 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
45,3571 s	46,3817 s	43,7859 s

Taula 3: Taula comparativa de temps de programa amb *feed_input()* paral·lelitzat amb 12 threads.

Com es pot observar a les taules, paral·lelitzar aquesta funció no aporta un benefici, de fet perjudica el programa i el fa més lent perquè es tarda més a crear tots els threads i dividir la càrrega de treball que en executar el treball.

Per aquest motiu, no s'aplicarà la paral·lelització a aquesta funció inicialment, però es considerarà afegir la paral·lelització a nivell global, per veure si combinar aquesta paral·lelització amb la paral·lelització d'altres funcions pot ser útil en la secció `Anàlisi de rendiment`.

2.2 Paral·lelització de *init_deltas()*

Aquesta és una funció que presenta diversos bucles paral·lelitzables, tot i que cal tenir en compte alguns matisos.

El primer *for* es pot paral·lelitzar, però no resulta convenient fer-ho directament amb `#pragma omp parallel for`. El motiu és que, si per exemple utilitzem 4 threads i paral·lelitzem el primer *for*, repartirem les 4 iteracions del bucle entre els 4 threads. Aquesta distribució pot ser perjudicial per al temps d'execució, ja que cada iteració del bucle exterior executa dos bucles interiors amb una càrrega de treball diferent.

Per exemple, en el segon bucle interior, quan `i = 0`, es realitzen

$$1024 \cdot 500 = 512000$$

assignacions, mentre que en l'última iteració, quan `i = 3`, només es realitzen

$$135 \cdot 10 = 1350$$

assignacions. Això provocaria un repartiment molt desequilibrat del treball, ja que alguns threads acabarien molt abans que altres i haurien d'esperar que finalitzés el thread amb més càrrega.

Per aquest motiu, utilitzem `#pragma omp parallel` per crear una única regió paral·lela en la qual es manté el mateix equip de threads durant l'execució dels diferents bucles. D'aquesta manera, evitem crear diverses regions paral·leles independents. Si paral·lelitzéssim per separat els dos bucles interiors amb `#pragma omp parallel for`, tindríem 8 regions paral·leles en total, ja que el bucle exterior té 4 iteracions i dins de cadascuna hi ha 2 bucles paral·lelitzats. Això introduiria una sobrecàrrega addicional associada a l'entrada i sortida de les regions paral·leles.

Pel que fa als bucles interiors, en el primer utilitzem `#pragma omp for nowait`. Aquest bucle realitza assignacions sobre el vector `dbias`, amb un nombre d'iteracions que va des de 1024 en la primera capa fins a 135 en l'última iteració del bucle exterior.

Amb `#pragma omp for` aconseguim repartir les iteracions del bucle entre els threads de la regió paral·lela. A més, amb la clàusula `nowait` eliminem la barrera de sincronització implícita que hi ha al final del bucle. D'aquesta manera, quan un thread acaba les iteracions que té assignades, pot continuar cap al següent bucle sense haver d'esperar que els altres threads acabin.

Això és possible perquè els dos bucles són independents: el primer inicialitza el vector `dbias`, mentre que el segon inicialitza el vector `dw`. Per tant, no existeixen dependències de dades entre aquestes dues operacions. Amb aquesta modificació podem reduir part del temps dedicat a la sincronització.

De totes maneres, com que les operacions del primer bucle són simplement assignacions a zero, totes les iteracions tenen pràcticament el mateix cost computacional. Per aquest motiu, és probable que els threads acabin les seves iteracions en temps semblants i que la millora obtinguda amb `nowait` sigui petita.

D'altra banda, en el segon bucle interior utilitzem `#pragma omp for schedule(static)`. Aquest és el bucle amb més càrrega de treball, especialment en la primera iteració del bucle exterior, on es realitzen 512000 assignacions.

Hem escollit `schedule(static)` perquè totes les iteracions tenen pràcticament el mateix cost computacional. D'aquesta manera, OpenMP reparteix les iteracions de forma estàtica i equilibrada entre els threads, sense necessitat d'assignar nous blocs de treball dinàmicament. Això permet reduir la sobrecàrrega associada a la planificació de les iteracions.

El codi paral·lelitzat és el següent:

```
1  void init_deltas() {
2      #pragma omp parallel
3      {
4          for (int i = 0; i < num_layers - 1; i++) {
5
6              #pragma omp for nowait
7              for (int j = 0; j < num_neurons[i]; j++)
8                  lay[i].dbias[j] = 0.0;
9
10             #pragma omp for schedule(static)
11             for (int j = 0; j < num_neurons[i + 1] *
                   num_neurons[i]; j++)
12                 lay[i].dw[j] = 0.0;
13         }
14     }
15 }
```

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *init_deltas()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40.7261 s	41.0380 s	40.7334 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
42.4691 s	42.4647 s	40.7182 s

Taula 4: Taula comparativa de temps de programa amb *init_deltas()* paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40.7261 s	40.7826 s	42.5414 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
43.0108 s	40.7396 s	40.8045 s

Taula 5: Taula comparativa de temps de programa amb *init_deltas()* paral·lelitzat amb 8 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40.7261 s	40.8568 s	40.8120 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
43.2060 s	42.6350 s	40.8100 s

Taula 6: Taula comparativa de temps de programa amb *init_deltas()* paral·lelitzat amb 12 threads.

Com es veu en les taules, no hem aconseguit una millora de rendiment respecte de l'execució seqüencial.

Una possible explicació és que aquests bucles únicament realitzen assignacions del valor zero als elements dels vectors, de manera que cada iteració té un cost computacional molt reduït. En aquestes condicions, la sobrecàrrega associada a la gestió dels threads, al repartiment del treball i a les sincronitzacions pot superar el temps estalviat gràcies a la paral·lelització. A més, el rendiment també pot estar limitat per la velocitat d'accés i escriptura a memòria.

Per tant, en aquest cas, paral·lelitzar els bucles no garanteix una reducció del temps d'execució, i els resultats experimentals indiquen que la versió seqüencial resulta més eficient. De totes maneres, comprovarem si combinar aquesta paral·lelització amb la paral·lelització d'altres funcions pot ser útil en la secció `Anàlisi de rendiment`.

2.3 Paral·lelització de *forward_prop()*

En aquesta funció, dels tres bucles, un és paral·lelitzable, un requereix consideracions addicionals i un presenta dependències entre iteracions

El *for* d'índex i no és paral·lelitzable, ja que una iteració usa les dades de la iteració anterior, és a dir hi ha una dependència clara entre iteracions del bucle. Aquesta dependència es fa explícita dins el *for* d'índex k, on s'usa `lay[i-1].actv[k]`.

El *for* d'índex j és paral·lelitzable perquè una iteració és completament independent de la següent i de l'anterior.

El *for* d'índex k és paral·lelitzable amb un matís, seria possible sempre que es controli l'accés a la variable `lay[i].z[j]` de manera correcta. Això es podria fer amb un `critical` o un `atomic`, però causaria un possible empitjorament del rendiment, ja que els threads estarien tota l'estona esperant per entrar a la regió crítica.

La manera més correcta és usar `reduction`, però a cada thread li correspondria una càrrega de treball molt petita (un producte i una suma), per la qual cosa suposem que la paral·lelització niuada seria contraproduent.

Per aquest motiu, inicialment es descarta paral·lelitzar aquest bucle, però es revisarà en l'apartat `Anàlisi de rendiment` per assegurar la nostra suposició.

La funció paral·lelitzada al bucle d'índex j queda de la següent manera:

```
1  void forward_prop() {
2      #pragma omp parallel
3      for (int i = 1; i < num_layers; i++) {
4          if (debug == 1) printf("Valors activacio capa %d \n"
               , i);
5          #pragma omp for schedule(static)
6          for (int j = 0; j < num_neurons[i]; j++) {
7              lay[i].z[j] = lay[i].bias[j];
8              for (int k = 0; k < num_neurons[i - 1]; k++)
9                  lay[i].z[j] +=
10                     ((lay[i - 1].out_weights[j * num_neurons
                           [i - 1] + k]) *
11                      (lay[i - 1].actv[k]));
12             if (i < num_layers - 1)  // Relu Activation
                   Function for Hidden Layers
13                 lay[i].actv[j] = ((lay[i].z[j]) < 0) ? 0 :
                       lay[i].z[j];
14             else  // Sigmoid Activation Function for Output
                   Layer
15                 lay[i].actv[j] = 1 / (1 + exp(-lay[i].z[j]))
                       ;
16             if (debug == 1) printf("%f %f ", lay[i].actv[j],
                    lay[i].z[j]);
17         }
```

```
18            if (debug == 1) printf("\n\n");
19        }
20    }
```

S'ha usat el `schedule(static)` per obtenir un baseline, ja que és la política estàndard d'OpenMP. Es consideraran altres polítiques en la part d'`Anàlisi de rendiment`.

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *forward_prop()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,6889 s	26,3509 s	26,3722 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
26,3609 s	26,7797 s	28,2628 s

Taula 7: Taula comparativa de temps de programa amb *forward_prop()* paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,6889 s	23,4829 s	23,3459 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
23,4431 s	24,5725 s	23,5300 s

Taula 8: Taula comparativa de temps de programa amb *forward_prop()* paral·lelitzat amb 8 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,6889 s	22,9876 s	23,0159 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
24,1328 s	23,9703 s	26,8209 s

Taula 9: Taula comparativa de temps de programa amb *forward_prop()* paral·lelitzat amb 12 threads.

Els temps d'execució del programa han millorat considerablement. Aparentment, la paral·lelització d'aquesta funció afecta positivament, falta, però acabar d'ajustar el nombre de threads ideal, si val la pena paral·lelitzar el bucle d'índex k, i també seria interessant considerar altres polítiques de divisió de càrrega de treball.

Addicionalment, considerar si posar un llindar mínim al nombre de neurones per tal de paral·lelitzar pot ser una idea que valgui la pena explorar.

En conclusió, aquesta funció sí que es paral·lelitza. Es pren com a base la paral·lelització descrita en aquest apartat i s'analitzaran els ítems esmentats a l'apartat d'`Anàlisi de rendiment`.

2.4 Paral·lelització de *compute_loss()*

Aquesta és una funció que conté un únic bucle. Per paral·lelitzar-la utilitzarem la directiva `#pragma omp parallel for private(p) reduction(+:tmpcost) schedule(static)`.

Aquesta directiva ens permet realitzar diverses operacions. En primer lloc, tenim un *for* de 10 iteracions. Per paral·lelitzar-lo, utilitzem `#pragma omp parallel for`, que s'encarrega de distribuir les iteracions del bucle entre els diferents threads disponibles.

D'altra banda, necessitem la clàusula `private(p)`, ja que `p` és una variable declarada fora del bucle que, per defecte, seria compartida entre tots els threads. Com que en cada iteració se li assigna un valor diferent, si no utilitzéssim `private(p)`, podria passar que diversos threads modifiquessin simultàniament la mateixa variable, provocant una condició de carrera (*race condition*) i, en conseqüència, resultats incorrectes. Amb aquesta clàusula, cada thread disposa de la seva pròpia còpia de la variable `p`, evitant així possibles conflictes.

Seguidament, la clàusula `reduction(+:tmpcost)` és essencial en aquesta paral·lelització. Això és degut al fet que `tmpcost` és una variable que acumula els resultats de les diferents iteracions mitjançant una suma. Si diversos threads modifiquessin aquesta variable simultàniament sense cap mecanisme de protecció, es podrien perdre actualitzacions i obtenir un resultat incorrecte.

Amb `reduction(+:tmpcost)`, aconseguim que cada thread disposi d'una còpia privada de `tmpcost`, inicialitzada a zero, en la qual acumula els resultats de les iteracions que té assignades. Un cop finalitzat el bucle, OpenMP combina totes aquestes sumes parcials per obtenir el valor final de `tmpcost`. D'aquesta manera, evitem les condicions de carrera i garantim que es tinguin en compte les contribucions de totes les iteracions.

Finalment, hem utilitzat `schedule(static)` per distribuir les iteracions del bucle de manera estàtica i equilibrada entre els threads. Hem escollit aquesta opció en lloc de `schedule(dynamic)` perquè totes les iteracions realitzen pràcticament les mateixes operacions i, per tant, tenen un cost computacional similar. D'aquesta manera, no és necessari realitzar un repartiment dinàmic del treball, que introduiria una sobrecàrrega addicional.

A més, encara que totes les iteracions tenen un cost computacional similar, la funció només realitza 10 iteracions per crida. Per tant, és possible que la sobrecàrrega associada a la gestió de la regió paral·lela, al repartiment del treball i a la reducció dels resultats parcials sigui superior al temps estalviat gràcies a la paral·lelització.

Per aquest motiu, és probable que aquesta paral·lelització no proporcioni una millora significativa del temps d'execució i que, fins i tot, pugui empitjorar el rendiment respecte de la versió seqüencial.

```
float compute_loss(int i) {
    double tmpcost = 0, p;

    #pragma omp parallel for private(p) reduction(+:tmpcost)
        schedule(static)
    for (int j = 0; j < num_neurons[num_layers - 1]; j++) {
        p = lay[num_layers - 1].actv[j];

        if (lay[num_layers - 1].actv[j] < EPSILON)
            p = EPSILON;
        else if (lay[num_layers - 1].actv[j] > 1 - EPSILON)
            p = 1 - EPSILON;

        tmpcost += desired_outputs[i][j] * log(p)
                + (1 - desired_outputs[i][j]) * log(1.0 - p
                    );
    }

    return -tmpcost / num_neurons[num_layers - 1];
}
```

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *compute_loss()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40.6208 s	41.1932 s	41.5916 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
44.0686 s	43.5493 s	41.2481 s

Taula 10: Taula comparativa de temps de programa amb textitcompute_loss() paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40.6208 s	42.0004 s	41.9586 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
44.4558 s	44.3085 s	41.8578 s

Taula 11: Taula comparativa de temps de programa amb *compute_loss()* paral·lelitzat amb 8 threads.

Execució seqüencial	Execució paral·lela 1	Execució paral·lela 2
40.6208 s	42.7304 s	42.8249 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
45.1416 s	44.8602 s	43.2013 s

Taula 12: Taula comparativa de temps de programa amb *compute_loss()* paral·lelitzat amb 12 threads.

Es pot observar com el rendiment empitjora i, en alguns casos, considerablement. Aquests resultats indiquen que la sobrecàrrega associada a la gestió de les regions paral·leles, al repartiment del treball i a la reducció dels resultats parcials pot superar el temps estalviat gràcies a la paral·lelització. Això provoca que el temps d'execució sigui superior al de la versió seqüencial i ens permet concloure que aquesta paral·lelització, per si sola, no resulta beneficiosa.

De totes maneres, abans de prendre una decisió definitiva, comprovarem si, en combinar aquesta funció amb altres funcions paral·lelitzades, el rendiment global del programa millora.

Per tant, encara que aquesta no sigui una funció amb la qual esperem reduir significativament el temps d'execució mitjançant la paral·lelització, encara no podem descartar-la completament sense analitzar-ne l'impacte en el rendiment global del programa.

2.5 Paral·lelització de *back_prop()*

La funció *back_prop()* és paral·lelitzable en alguns dels bucles que conté.

El primer i segon bucle són paral·lelitzables, tot i que poden no suposar un gran avantatge, ja que només es fa una inicialització a 0 en el primer bucle, i en el segon es fa un producte escalar. Per intentar mitigar el cost d'aquestes paral·lelitzacions, es crea una regió paral·lela que engloba tota la funció.

El tercer *for* no és paral·lelitzable directament, perquè es fa una suma acumulada a `.dactv[]`. Caldria gestionar l'accès a aquesta variable, amb un `atomic` o `critical` que causaria un empitjorament del rendiment per l'espera que estarien obligats a fer els threads. Altre cop, la manera correcta seria usar `reduction`, però requeriria de crear un vector local on posar els productes escalars i creiem que seria massa complicació de moment.

Tot i això es revisarà si paral·lelitzar aquest bucle és positiu en l'apartat `Anàlisi de rendiment`.

El quart *for* no és paral·lelitzable al complet, només es poden paral·lelitzar alguns dels bucles interiors. El primer bucle interior, d'índex k que es troba dins el `if(i > 1)`, sí és paral·lelitzable, amb el mateix matís que els 2 primers bucles. Al segon bucle, d'índex j, li passa el mateix que al bucle anterior d'índex

j, sí és paral·lelitzable però requeriex del control d'accès a un vector, usant `reduction` per fer-ho correctament.

Aquest punt també es deixa per revisar en l'apartat `Anàlisi de rendiment`.

La funció paral·lelitzada queda de la següent manera:

```
void back_prop(int p) {
    #pragma omp parallel
    {
        // Output Layer
        #pragma omp for schedule(static)
        for (int k = 0; k < num_neurons[num_layers - 2]; k
            ++)
                lay[num_layers - 2].dactv[k] = 0;

        #pragma omp for schedule(static)
        for (int j = 0; j < num_neurons[num_layers - 1]; j
            ++) {
                lay[num_layers - 1].dz[j] = (lay[num_layers
                    - 1].actv[j] - desired_outputs[p][j]) * (
                    lay[num_layers - 1].actv[j]) * (1 - lay[
                    num_layers - 1].actv[j]);
                lay[num_layers - 1].dbias[j] += lay[
                    num_layers - 1].dz[j];
        }

        #pragma omp single
        for (int j = 0; j < num_neurons[num_layers - 1]; j
            ++) {
            for (int k = 0; k < num_neurons[num_layers - 2];
                 k++) {
                lay[num_layers - 2].dw[j * num_neurons[
                    num_layers - 2] + k] += (lay[num_layers -
                     1].dz[j] * lay[num_layers - 2].actv[k]);
                lay[num_layers - 2].dactv[k] += lay[
                    num_layers - 2].out_weights[j *
                    num_neurons[num_layers - 2] + k] * lay[
                    num_layers - 1].dz[j];
            }
        }

        // Hidden Layers
        for (int i = num_layers - 2; i > 0; i--) {
            if ( i > 1 )
                #pragma omp for schedule(static)
                for (int k = 0; k < num_neurons[i - 1]; k++)
                     {
                    lay[i - 1].dactv[k] = 0;
                }
            #pragma omp single
```

```
31              for (int j = 0; j < num_neurons[i]; j++) {
32                  lay[i].dz[j] = (lay[i].z[j] >= 0) ? lay[i].
                        dactv[j] : 0;
33
34                  for (int k = 0; k < num_neurons[i - 1]; k++)
                         {
35                      lay[i - 1].dw[j * num_neurons[i - 1] + k
                            ] += lay[i].dz[j] * lay[i - 1].actv[k
                            ];
36
37                      if (i > 1)
38                          lay[i - 1].dactv[k] += lay[i - 1].
                                out_weights[j * num_neurons[i -
                                1] + k] * lay[i].dz[j];
39                  }
40                  lay[i].dbias[j] += lay[i].dz[j];
41              }
42          }
43      }
44  }
```

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *back_prop()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,6724 s	27.6823 s	27.4227 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
30.2373 s	27.4860 s	33.7570 s

Taula 13: Taula comparativa de temps de programa amb *back_prop()* paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,6724 s	30.3219 s	28.6657 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
28.9451 s	28.6156 s	30.0278 s

Taula 14: Taula comparativa de temps de programa amb *back_prop()* paral·lelitzat amb 8 threads.

Execució seqüencial	Execució paral·lela 1	Execució paral·lela 2
40,6724 s	29.0796 s	29.4674 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
29.7323 s	30.1132 s	29.6209 s

Taula 15: Taula comparativa de temps de programa amb *back_prop()* paral·lelitzat amb 12 threads.

Es veu que la paral·lelització ha sigut profitosa en un àmbit, disminuint el temps d'execució del programa.

Aquesta funció sí que es paral·lelitzarà, ja que produeix una disminució de temps significativa. Es pren com a base la paral·lelització descrita en aquest apartat i s'analitzaran certes millores a l'apartat d'`Anàlisi de rendiment`.

2.6 Paral·lelització de *update_weights()*

Aquí la funció és clarament paral·lelitzable, la funció té únicament bucles i assignacions, no té ni condicionals ni sumes múltiples.

El primer bucle, d'índex i, no el paral·lelitzarem perquè té poques iteracions i amb càrrega desigual, però els bucles interiors sí que es paral·lelitzen. El primer *for* interior i el segon es poden paral·lelitzar perfectament.

Ambdues paral·lelitzacions poden usar la clàusula **nowait** perquè els dos bucles d'índex j són independents l'un de l'altre, així que no és necessari crear una barrera en acabar el primer *for*.

El codi paral·lelitzat de la funció és el següent:

```
1  void update_weights(void) {
2      #pragma omp parallel
3      for (int i = 0; i < num_layers - 1; i++) {
4          #pragma omp for schedule(static) nowait
5          for (int j = 0; j < num_neurons[i + 1]; j++)
6              for (int k = 0; k < num_neurons[i]; k++)  //
                   Update Weights
7                  lay[i].out_weights[j * num_neurons[i] + k] =
8                      (lay[i].out_weights[j * num_neurons[i] +
                           k]) -
9                      (alpha * lay[i].dw[j * num_neurons[i] +
                           k]);
10
11         #pragma omp for schedule(static) nowait
12         for (int j = 0; j < num_neurons[i]; j++)  // Update
               Bias
13             lay[i].bias[j] = lay[i].bias[j] - (alpha * lay[i
                   ].dbias[j]);
14     }
```

```
15 }
```

A continuació es mostren unes taules comparatives del temps d'execució del programa amb únicament la funció *update_weights()* paral·lelitzada, amb diversos nombres de threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,7722 s	40,6515 s	43,0666 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
42,7429 s	40,6589 s	40,6043 s

Taula 16: Taula comparativa de temps de programa amb *update_weights()* paral·lelitzat amb 4 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,7722 s	40,9170 s	40,6025 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
42,8321 s	42,8739 s	40,6488 s

Taula 17: Taula comparativa de temps de programa amb *update_weights()* paral·lelitzat amb 8 threads.

Execució seqüencial	**Execució paral·lela 1**	**Execució paral·lela 2**
40,7722 s	40,7631 s	40,7122 s
Execució paral·lela 3	**Execució paral·lela 4**	**Execució paral·lela 5**
43,2007 s	42,9676 s	40,6938 s

Taula 18: Taula comparativa de temps de programa amb *update_weights()* paral·lelitzat amb 12 threads.

Les dades no mostren una clara millora en paral·lelitzar la funció, les execucions paral·leles tenen aproximadament el mateix temps que la seqüencial, llevat d'una o dues execucions per taula, el que pot indicar una variabilitat més elevada.

Paral·lelitzar o no aquesta funció no sembla una decisió tan clara, així que inicialment es decideix treballar amb la funció paral·lelitzada, però es revisarà si el fet de no paral·lelitzar suposa una millora global a l'apartat d'`Anàlisi de rendiment`.

3 Anàlisi de rendiment

En aquesta secció s'estudia el rendiment del programa variant petits paràmetres de les funcions paral·lelitzades. Es farà primer un estudi a escala de funció per acabar de determinar quines són les millors clàusules en aquella funció i a continuació un estudi del programa a nivell global (del fitxer *training.c* que és el que correspon aquest apartat) per acabar d'ajustar decisions globals.

No es considera un apartat per les funcions *init_deltas()*, *update_weights()* i *compute_loss()* perquè la decisió és des del punt de vista global sobre si es paral·lelitza o no, s'estudiarà en l'apartat global.

Al llarg de les següents seccions es farà ús dels conceptes *Speedup* i *Eficiència*, els quals definim a continuació.

Definició 3.1 (Speedup) *El **speedup** (S_p) mesura quant més ràpid s'executa una tasca utilitzant p processadors en comparació amb l'execució seqüencial tradicional amb un sol processador.*

Es defineix com:

$$S_p = \frac{T_1}{T_p}$$

Definició 3.2 (Eficiència) *L'**eficiència** (E_p) mesura com de bé s'estan aprofitant els recursos de càlcul. Indica quin percentatge del temps total dels p processadors s'està invertint realment a calcular.*

Es defineix com:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p}$$

3.1 Fine-tuning de *forward_prop()*

De la funció *forward_prop()* s'ha decidit revisar els següents paràmetres:

- Paral·lelització del bucle d'índex k.
- Altres polítiques de divisió de càrrega: **dynamic**, **guided**.

Per iniciar el *fine-tuning* s'analitza el rendiment de la funció paral·lelitzada amb la paral·lelització afegida del bucle d'índex k, amb un nombre de threads d'1 a 12.

Per a considerar la paral·lelització del bucle d'índex k s'ha de fer una lleugera modificació del codi base, per tal de crear una variable a la qual aplicar la reducció. I també se'ns ha generat un altre problema, que és que no es pot paral·lelitzar un *for* dins un altre *for* ja paral·lelitzat. Per solucionar aquest problema s'ha usat la directiva `#pragma omp simd` d'OpenMP, la qual s'ha hagut de recercar per compte propi i queda explicat el seu funcionament en l'apartat `Concepte de SIMD`.

El segment de la funció modificada és el següent:

```
void forward_prop() {
    #pragma omp parallel
    for (int i = 1; i < num_layers; i++) {
        if (debug == 1) printf("Valors activacio capa %d \n"
            , i);
        #pragma omp for schedule(static)
        for (int j = 0; j < num_neurons[i]; j++) {
            float z = lay[i].bias[j];
            #pragma omp simd reduction(+:z)
            for (int k = 0; k < num_neurons[i - 1]; k++)
                z += ((lay[i - 1].out_weights[j *
                    num_neurons[i - 1] + k]) * (lay[i - 1].
                    actv[k]));

            lay[i].z[j] = z;
            if (i < num_layers - 1)  // Relu Activation
                Function for Hidden Layers
                lay[i].actv[j] = ((lay[i].z[j]) < 0) ? 0 :
                    lay[i].z[j];
            else  // Sigmoid Activation Function for Output
                Layer
                lay[i].actv[j] = 1 / (1 + exp(-lay[i].z[j]))
                    ;
            if (debug == 1) printf("%f %f ", lay[i].actv[j],
                 lay[i].z[j]);
        }
        if (debug == 1) printf("\n\n");
    }
}
```

Es recullen les dades en una taula amb la mitjana i desviació del temps del programa, el speedup i l'eficiència al llarg de 10 execucions per nombre de thread, amb execucions seqüencials de referència.

Fils	**Temps (s)**	**Speedup**	**Eficiència**
seq	28,869 ± 0,807	1	—
1	41,245 ± 1,021	0,70 ± 0,03	0,70
2	30,752 ± 0,369	0,94 ± 0,03	0,47
3	27,496 ± 0,548	1,05 ± 0,04	0,35
4	25,795 ± 0,477	1,12 ± 0,04	0,28
5	24,654 ± 0,353	1,17 ± 0,04	0,23
6	23,965 ± 0,150	1,20 ± 0,03	0,20
7	23,536 ± 0,358	1,23 ± 0,04	0,18
8	23,344 ± 0,535	1,24 ± 0,04	0,15
9	22,999 ± 0,206	1,26 ± 0,04	0,14
10	22,896 ± 0,379	1,26 ± 0,04	0,13
11	22,950 ± 0,507	1,26 ± 0,04	0,11
12	23,137 ± 0,471	1,25 ± 0,04	0,10

Taula 19: Mitjana ± desviació estàndard de les execucions de *forward_prop()* amb la paral·lelització del bucle d'índex k.

Una qüestió sorprenent que s'ha vist en aquestes execucions és que el temps d'execució del programa en seqüencial s'ha reduït de manera substancial, inclús és més ràpid que la paral·lelització amb 2 threads. Una possible explicació d'aquest fet és que s'ha creat una variable local z i, per tant no s'han de gestionar els accessos a memòria d'un vector compartit, que és molt més costòs.

Amb aquesta paral·lelització afegida es pot veure que el temps millora en general, així que afegir la paral·lelització del *for* d'índex k és beneficiós per la velocitat del programa. El millor temps es troba amb 10 threads.

Fils	**Temps (s)**	**Diferència (s)**	**Diferència (%)**
seq	28,869	5,973	26,09
1	41,245	18,349	80,14
2	30,752	7,856	34,31
3	27,496	4,600	20,09
4	25,795	2,899	12,66
5	24,654	1,758	7,68
6	23,965	1,069	4,67
7	23,536	0,640	2,80
8	23,344	0,448	1,96
9	22,999	0,103	0,45
10	22,896	0,000	0,00
11	22,950	0,054	0,24
12	23,137	0,241	1,05

Taula 20: Diferència del temps mitjà de cada configuració respecte al millor temps (10 fils, 22,896 s) amb la paral·lelització del bucle d'índex k a *forward_prop()*.

Amb aquesta taula podem escollir una tolerància amb la que triar un interval òptim de threads per a la funció, usant una tolerància del 2% triaríem l'interval $[8, 10]$ com l'interval òptim de threads.

El següent punt a ajustar de la funció és la política de divisió de càrrega que s'usa. Es pren com a base la millor configuració de la funció trobada fins ara, és a dir, amb el bucle d'índex k paral·lelitzat. Per a aquest paràmetre no es faran execucions amb diversos nombres de threads, sinó que s'agafa el millor nombre de threads trobat fins ara (10 threads) pel nombre de combinacions que generaria considerar tots els nombres de threads.

Les dues polítiques que falten per provar són `dynamic` i `guided`. Ambdues permeten el paràmetre `chunk`, que determina coses diferents segons quina política s'usi. Per a **dynamic**, el **chunk** determina la mida dels blocs d'iteracions que es donen a cada thread. A **guided**, els blocs d'iteracions són inicialment més grans, i van disminuint fins a arribar a la mida definida a **chunk**.

S'usaran els valors de $1, 2, 4, 8, 16$ per al paràmetre `chunk`.

Política	Chunk	Temps (s)	Dif. respecte *static* (%)
static	—	**22,812 ± 0,186**	**0,00**
dynamic	1	25,431 ± 1,304	11,48
	2	24,521 ± 0,966	7,49
	4	23,952 ± 1,035	5,00
	8	23,711 ± 0,477	3,94
	16	24,098 ± 0,699	5,63
guided	1	23,957 ± 0,910	5,02
	2	23,801 ± 0,685	4,33
	4	24,216 ± 0,822	6,15
	8	23,992 ± 0,623	5,17
	16	24,393 ± 1,166	6,93

Taula 21: Temps d'execució segons la política i la mida de *chunk* amb 10 fils (mitjana ± desviació estàndard).

Les execucions mostren que aplicar una política de divisió de càrrega que no sigui `static` és contraproduent, així que es mantindrà el `schedule(static)`.

Finalment, la funció *forward_prop()* paral·lelitzada queda de la següent manera:

```
void forward_prop() {
    #pragma omp parallel
    for (int i = 1; i < num_layers; i++) {
        if (debug == 1) printf("Valors activacio capa %d \n"
            , i);
        #pragma omp for schedule(static)
        for (int j = 0; j < num_neurons[i]; j++) {
            float z = lay[i].bias[j];
            #pragma omp simd reduction(+:z)
            for (int k = 0; k < num_neurons[i - 1]; k++)
                z += ((lay[i - 1].out_weights[j *
                    num_neurons[i - 1] + k]) * (lay[i - 1].
                    actv[k]));

            lay[i].z[j] = z;
            if (i < num_layers - 1)  // Relu Activation
                Function for Hidden Layers
                lay[i].actv[j] = ((lay[i].z[j]) < 0) ? 0 :
                    lay[i].z[j];
            else  // Sigmoid Activation Function for Output
                Layer
                lay[i].actv[j] = 1 / (1 + exp(-lay[i].z[j]))
                    ;
            if (debug == 1) printf("%f %f ", lay[i].actv[j],
                 lay[i].z[j]);
        }
        if (debug == 1) printf("\n\n");
    }
```

```
}
```

3.2 Fine-tuning de *back_prop()*

De la funció *back_prop()* s'han decidit revisar els següents paràmetres:

- Paral·lelització del tercer *for*.
- Paral·lelització del bucle d'índex j dins el quart bucle.
- Altres polítiques de divisió de càrrega: **dynamic**, **guided**.

Per iniciar el *fine-tuning* d'aquesta funció, es considera paral·lelitzar el tercer bucle de la funció, que inicialment s'ha gestionat amb un `#pragma omp single`, el qual hem utilitzat ja que, si no ho fèiem, diversos threads entraven en una condició de carrera i disminuïa el nombre d'encerts. Això fa que, amb la directiva `#pragma omp single`, el nombre d'encerts sigui el correcte, però el càlcul d'aquesta part continuï sent seqüencial, amb la sobrecàrrega addicional associada a la paral·lelització.

Si es vol paral·lelitzar correctament aquest bucle, cal controlar l'accés a les variables on es fa l'acumulació de la suma. Al primer terme, corresponent a `lay[].dw[]`, no cal aplicar-hi la clàusula `reduction`, ja que cada parell (j, k) identifica una posició única del vector i, per tant, no es genera el problema d'accés concurrent que es produeix amb `lay[].dactv[]`, on sí que cal aplicar `reduction`.

Per poder aplicar aquesta clàusula, es pot crear un punter `float *dactv` que apunti a `lay[num_layers - 2].dactv`, i aplicar `reduction(+:dactv[:num_neurons[num_layers - 2]])`. Això permet disposar d'un punter compartit entre tots els threads que referencia el vector sobre el qual volem realitzar la reducció. En aquest cas, protegim les acumulacions sobre el vector `dactv` de la capa `num_layers - 2`, que correspon a la capa 3, ja que tenim `num_layers = 5` i $5 - 2 = 3$. Sabem que la capa 3 conté 135 neurones, de manera que es crea una còpia privada d'un vector de 135 elements per a cada thread. Finalment, OpenMP s'encarrega d'acumular les contribucions de cadascuna d'aquestes còpies privades sobre el vector original.

El codi queda:

```
void back_prop(int p) {
    float *dactv = lay[num_layers - 2].dactv;

    #pragma omp parallel
    {
        // Output Layer
        #pragma omp for schedule(static) nowait
```

```
8           for (int k = 0; k < num_neurons[num_layers - 2]; k
                ++)
9               lay[num_layers - 2].dactv[k] = 0;
10
11          #pragma omp for schedule(static)
12          for (int j = 0; j < num_neurons[num_layers - 1]; j
                ++) {
13              lay[num_layers - 1].dz[j] = (lay[num_layers
                    - 1].actv[j] - desired_outputs[p][j]) * (
                    lay[num_layers - 1].actv[j]) * (1 - lay[
                    num_layers - 1].actv[j]);
14              lay[num_layers - 1].dbias[j] += lay[
                    num_layers - 1].dz[j];
15          }
16
17          #pragma omp for collapse(2) schedule(static)
                reduction(+:dactv[:num_neurons[num_layers - 2]])
18          for (int j = 0; j < num_neurons[num_layers - 1]; j
                ++) {
19              for (int k = 0; k < num_neurons[num_layers - 2];
                     k++) {
20              lay[num_layers - 2].dw[j * num_neurons[
                    num_layers - 2] + k] += (lay[num_layers -
                     1].dz[j] * lay[num_layers - 2].actv[k]);
21
22              dactv[k] += lay[num_layers - 2].out_weights[
                    j * num_neurons[num_layers - 2] + k] *
                    lay[num_layers - 1].dz[j];
23          }
24          }
25
26          // Hidden Layers
27          for (int i = num_layers - 2; i > 0; i--) {
28              if ( i > 1 )
29                  #pragma omp for schedule(static)
30                  for (int k = 0; k < num_neurons[i - 1]; k++)
                         {
31                      lay[i - 1].dactv[k] = 0;
32                  }
33              #pragma omp single
34              for (int j = 0; j < num_neurons[i]; j++) {
35                  lay[i].dz[j] = (lay[i].z[j] >= 0) ? lay[i].
                        dactv[j] : 0;
36
37                  for (int k = 0; k < num_neurons[i - 1]; k++)
                         {
38                      lay[i - 1].dw[j * num_neurons[i - 1] + k
                            ] += lay[i].dz[j] * lay[i - 1].actv[k
                            ];
39
```

```
40                          if (i > 1)
41                              lay[i - 1].dactv[k] += lay[i - 1].
                                    out_weights[j * num_neurons[i -
                                    1] + k] * lay[i].dz[j];
42                      }
43                      lay[i].dbias[j] += lay[i].dz[j];
44                  }
45              }
46          }
47      }
```

Es recullen les dades en la mateixa taula, amb la mitjana i desviació del temps, speedup i eficiència al llarg de 10 execucions.

Fils	**Temps (s)**	**Speedup**	**Eficiència**
seq	28,199 ± 0,883	1	—
1	26,522 ± 1,018	1,06 ± 0,05	1,06
2	27,766 ± 1,594	1,02 ± 0,07	0,51
3	27,817 ± 1,040	1,01 ± 0,05	0,34
4	27,648 ± 0,559	1,02 ± 0,04	0,25
5	28,312 ± 1,154	1,00 ± 0,05	0,20
6	28,385 ± 0,494	0,99 ± 0,04	0,17
7	28,739 ± 0,557	0,98 ± 0,04	0,14
8	29,240 ± 0,573	0,96 ± 0,04	0,12
9	29,587 ± 0,920	0,95 ± 0,04	0,11
10	29,520 ± 0,920	0,96 ± 0,04	0,10
11	29,448 ± 0,454	0,96 ± 0,03	0,09
12	30,982 ± 0,706	0,91 ± 0,04	0,08

Taula 22: Mitjana ± desviació estàndard de les execucions de *back_prop()* amb paral·lelització a les *Output layers*.

Podem veure aquests resultats en els gràfics següents:

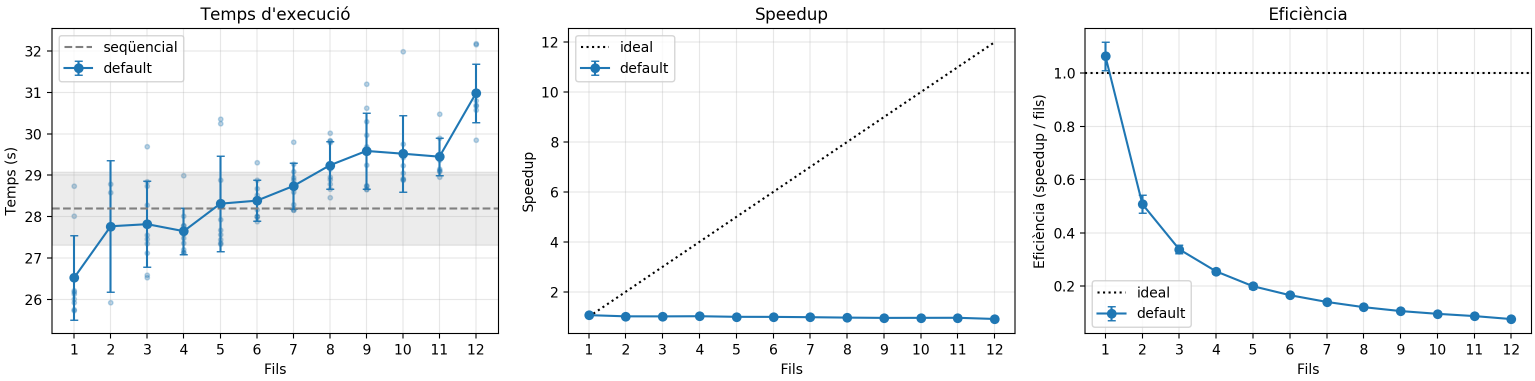


Figura 1: Comparació del temps d'execució, l'acceleració i l'eficiència segons el nombre de fils.

Segons les dades de la taula, aquesta paral·lelització, de moment, no ens genera un gran avantatge en el temps d'execució respecte de l'execució seqüencial. Si ens baséssim només en això, podríem considerar que no s'hauria d'aplicar, perquè no genera un guany de temps significatiu i el *speedup* és aproximadament 1, cosa que indica que el temps total d'execució pràcticament no millora en utilitzar més processadors.

De totes maneres, no podem prendre encara una decisió sobre si mantenir o no la paral·lelització, ja que encara tenim la part de les *Hidden Layers* sense paral·lelitzar de forma òptima. Això pot fer que, encara que ara la part de l'*Output Layer* sigui més ràpida, les *Hidden Layers* continuïn executant-se de manera pràcticament seqüencial, fet que pot provocar que la millora no sigui significativa en el temps total d'execució del programa.

La diferència percentual es pot observar en la taula següent.

Fils	Temps (s)	Diferència (s)	Diferència (%)
seq	28,199	1,677	6,32
1	26,522	0,000	0,00
2	27,766	1,244	4,69
3	27,817	1,295	4,88
4	27,648	1,126	4,25
5	28,312	1,790	6,75
6	28,385	1,863	7,02
7	28,739	2,217	8,36
8	29,240	2,718	10,25
9	29,587	3,065	11,56
10	29,520	2,998	11,30
11	29,448	2,926	11,03
12	30,982	4,460	16,82

Taula 23: Diferència del temps mitjà de cada configuració respecte al millor temps (1 fil, 26,522 s) amb la paral·lelització de les *Output layers* a *back_prop()*.

Es mantindrà però, perquè, tal com es veurà a continuació, millora el temps de la funció en conjunt a la següent paral·lelització.

El següent punt considera paral·lelitzar el bucle d'índex j de les *Hidden Layers*. La idea és aplicar una reducció sobre `lay[i - 1].dactv[k]`.

Abans, però, es fan algunes modificacions al codi per facilitar aquesta tasca, ja que, amb la manera com s'accedia inicialment a `lay[i - 1].dactv[k]`, no es podia aplicar directament la reducció amb la sintaxi que volíem utilitzar.

La primera modificació consisteix a crear un punter que apunti a `lay[i - 1].dactv` si `i > 1`, o a una variable auxiliar `dummy` si `i = 1`, ja que, en aquest últim cas, no és necessari acumular gradients d'activació a la capa anterior. D'aquesta manera, podem utilitzar el punter tant per inicialitzar el vector com per aplicar-hi la reducció.

L'assignació de l'adreça a la qual apunta el punter es fa de manera seqüencial per evitar que diversos threads modifiquin simultàniament el mateix punter compartit. Per aquest motiu, aquesta assignació es realitza dins d'una secció `#pragma omp single`, que també garanteix, mitjançant la barrera implícita final, que tots els threads disposin del punter correctament assignat abans de continuar.

Després, al bucle d'índex j, es pot aplicar la clàusula `reduction(+:dactv_hidden[...])`.

Queda de la següent manera el codi:

```
1  void back_prop(int p) {
2      float *dactv = lay[num_layers - 2].dactv;
3      float *dactv_hidden;
4      float dummy = 0.0f;
```

```
5
6       #pragma omp parallel
7       {
8           // Output Layer
9           #pragma omp for schedule(static) nowait
10          for (int k = 0; k < num_neurons[num_layers - 2]; k
                ++)
11              lay[num_layers - 2].dactv[k] = 0;
12
13          #pragma omp for schedule(static)
14          for (int j = 0; j < num_neurons[num_layers - 1]; j
                ++) {
15              lay[num_layers - 1].dz[j] = (lay[num_layers
                    - 1].actv[j] - desired_outputs[p][j]) * (
                    lay[num_layers - 1].actv[j]) * (1 - lay[
                    num_layers - 1].actv[j]);
16              lay[num_layers - 1].dbias[j] += lay[
                    num_layers - 1].dz[j];
17          }
18
19          #pragma omp for collapse(2) schedule(static)
                reduction(+:dactv[:num_neurons[num_layers - 2]])
20          for (int j = 0; j < num_neurons[num_layers - 1]; j
                ++) {
21              for (int k = 0; k < num_neurons[num_layers - 2];
                     k++) {
22              lay[num_layers - 2].dw[j * num_neurons[
                    num_layers - 2] + k] += (lay[num_layers -
                     1].dz[j] * lay[num_layers - 2].actv[k]);
23
24              dactv[k] += lay[num_layers - 2].out_weights[
                    j * num_neurons[num_layers - 2] + k] *
                    lay[num_layers - 1].dz[j];
25              }
26          }
27
28          // Hidden Layers
29          for (int i = num_layers - 2; i > 0; i--) {
30
31              #pragma omp single
32              {
33              dactv_hidden = (i > 1) ? lay[i - 1].dactv :
                    &dummy;
34              }
35
36              if ( i > 1 ){
37
38              #pragma omp for schedule(static)
39              for (int k = 0; k < num_neurons[i - 1]; k++)
                     {
```

```
40                      dactv_hidden[k] = 0;
41                  }
42              }
43
44              #pragma omp for schedule(static) reduction(+:
                    dactv_hidden[:(i > 1 ? num_neurons[i - 1] :
                    1)])
45              for (int j = 0; j < num_neurons[i]; j++) {
46                  lay[i].dz[j] = (lay[i].z[j] >= 0) ? lay[i].
                        dactv[j] : 0;
47
48                  for (int k = 0; k < num_neurons[i - 1]; k++)
                         {
49                      lay[i - 1].dw[j * num_neurons[i - 1] + k
                            ] += lay[i].dz[j] * lay[i - 1].actv[k
                            ];
50
51                      if (i > 1)
52
53                          dactv_hidden[k] += lay[i - 1].
                                out_weights[j * num_neurons[i -
                                1] + k] * lay[i].dz[j];
54                  }
55                  lay[i].dbias[j] += lay[i].dz[j];
56              }
57          }
58      }
59  }
```

Fils	Temps (s)	Speedup	Eficiència
seq	26,940 ± 1,264	1	—
1	25,996 ± 0,605	1,04 ± 0,05	1,04
2	16,814 ± 0,674	1,60 ± 0,10	0,80
3	13,693 ± 0,389	1,97 ± 0,11	0,66
4	12,053 ± 0,079	2,24 ± 0,11	0,56
5	11,280 ± 0,180	2,39 ± 0,12	0,48
6	10,741 ± 0,105	2,51 ± 0,12	0,42
7	10,510 ± 0,435	2,56 ± 0,16	0,37
8	10,209 ± 0,137	2,64 ± 0,13	0,33
9	10,177 ± 0,249	2,65 ± 0,14	0,29
10	10,122 ± 0,150	2,66 ± 0,13	0,27
11	10,162 ± 0,124	2,65 ± 0,13	0,24
12	10,472 ± 0,506	2,57 ± 0,17	0,21

Taula 24: Mitjana ± desviació estàndard de les execucions de *back_prop()* amb paral·lelització a les *Output layers* i *Hidden layers*.

Podem veure aquests resultats en els gràfics següents:

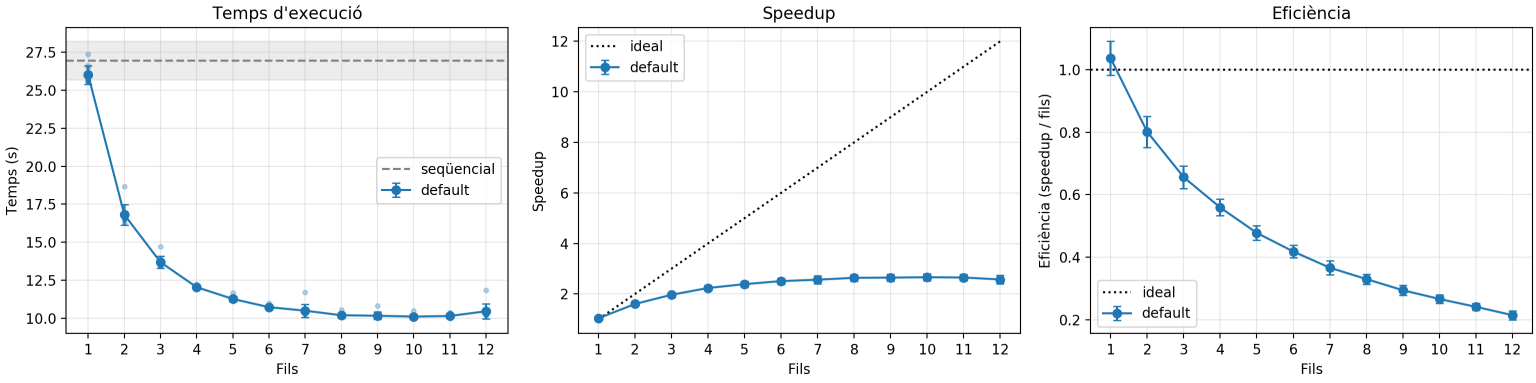


Figura 2: Comparació del temps d'execució, l'acceleració i l'eficiència segons el nombre de fils.

Ara es veu que aquesta paral·lelització sí aporta un gran benefici, amb una reducció substancial de temps i un *speedup* creixient fins als 10 threads.

També es pot comprovar en la taula percentual.

Fils	**Temps (s)**	**Diferència (s)**	**Diferència (%)**
seq	26,940	16,818	166,15
1	25,996	15,874	156,83
2	16,814	6,692	66,11
3	13,693	3,571	35,28
4	12,053	1,931	19,08
5	11,280	1,158	11,44
6	10,741	0,619	6,12
7	10,510	0,388	3,83
8	10,209	0,087	0,86
9	10,177	0,055	0,54
10	10,122	0,000	0,00
11	10,162	0,040	0,40
12	10,472	0,350	3,46

Taula 25: Diferència del temps mitjà de cada configuració respecte al millor temps (10 fils, 10,122 s) amb la paral·lelització de les *Output layers* i les *Hidden layers* a *back_prop()*.

Finalment, queda estudiar si canviar la política de distribució de treball d'OpenMP pot ajudar o és contraproduent. S'usa la mateixa metodologia que a

la funció *forward_prop()*, amb mides de `chunk` d'1, 2, 4, 8 i 16 per a `dynamic` i `guided`.

Es realitzen les execucions amb 10 threads, ja que és el nombre de threads que ha resultat amb millor temps.

Política	**Chunk**	**Temps (s)**	**Dif. respecte *static* (%)**
static	—	**10,107 ± 0,071**	**0,00**
dynamic	1	20,126 ± 0,082	99,12
	2	16,897 ± 0,094	67,17
	4	14,931 ± 0,241	47,73
	8	13,451 ± 0,061	33,08
	16	12,983 ± 0,342	28,45
guided	1	13,805 ± 0,164	36,59
	2	13,610 ± 0,156	34,65
	4	13,317 ± 0,095	31,76
	8	13,147 ± 0,130	30,07
	16	12,924 ± 0,080	27,87

Taula 26: Temps d'execució segons la política i la mida de *chunk* amb 10 fils (mitjana ± desviació estàndard).

Les dades recollides en la taula ens podrien indicar que augmentar la mida del `chunk` podria ser beneficiós, i anar reduint el temps d'execució, però una comprovació amb `chunk = 32` ha descartat aquesta possibilitat.

Per tant, la millor política de distribució és `schedule(static)`, i el codi paral·lelitzat de la funció és el descrit anteriorment. Això es deu al fet que les operacions que duen a terme els threads tenen un cost computacional similar, és a dir, no hi ha una gran diferència en el temps d'execució de les diferents iteracions. Per tant, utilitzar una política de distribució dinàmica acaba introduint una sobrecàrrega addicional a l'hora de repartir les iteracions entre els threads, fet que pot augmentar el temps total d'execució.

3.3 Fine-tuning global

Queda decidir sobre els següents ítems:

- Paral·lelitzar la funció *feed_input()*
- Paral·lelitzar la funció *init_deltas()*.
- Paral·lelitzar la funció *update_weights()*.
- Paral·lelitzar la funció *compute_loss()*.

És a dir, volem saber si les funcions que, paral·lelitzades per elles soles no aporten benefici, en conjunt sí que n'aporten.

Per comprovar-ho s'afegiran les funcions paral·lelitzades d'una en una sobre la base de `training.c` amb les funcions *forward_prop()* i *back_prop()* paral·lelitzades.

El temps des del que es parteix és el següent.

Fils	**Temps (s)**	**Speedup**	**Eficiència**
1	42,265 ± 0,671	1,00 ± 0,02	1,00
2	22,250 ± 0,263	1,90 ± 0,04	0,95
3	17,017 ± 0,544	2,48 ± 0,09	0,83
4	13,316 ± 0,232	3,17 ± 0,07	0,79
5	11,343 ± 0,119	3,73 ± 0,07	0,75
6	10,329 ± 0,168	4,09 ± 0,09	0,68
7	9,132 ± 0,047	4,63 ± 0,08	0,66
8	8,373 ± 0,085	5,05 ± 0,10	0,63
9	7,830 ± 0,039	5,40 ± 0,09	0,60
10	7,521 ± 0,070	5,62 ± 0,10	0,56
11	7,336 ± 0,091	5,76 ± 0,12	0,52
12	8,115 ± 1,262	5,21 ± 0,81	0,43

Taula 27: Mitjana ± desviació estàndard de les execucions amb les paral·lelitzacions ajustades de *forward_prop()* i *back_prop()*.

Es veu que la paral·lelització d'aquestes dues funcions ja aporta un gran benefici, si afegim la paral·lelització de la funció *feed_input()* obtenim els seguents temps.

Fils	**Temps (s)**	**Speedup**	**Eficiència**
1	40,635 ± 0,892	1,00 ± 0,03	1,00
2	21,325 ± 0,143	1,91 ± 0,04	0,95
3	14,836 ± 0,067	2,74 ± 0,06	0,91
4	11,543 ± 0,072	3,52 ± 0,08	0,88
5	9,688 ± 0,053	4,19 ± 0,09	0,84
6	8,701 ± 0,047	4,67 ± 0,11	0,78
7	7,768 ± 0,069	5,23 ± 0,12	0,75
8	7,194 ± 0,043	5,65 ± 0,13	0,71
9	6,762 ± 0,033	6,01 ± 0,14	0,67
10	6,580 ± 0,032	6,18 ± 0,14	0,62
11	6,466 ± 0,046	6,28 ± 0,15	0,57
12	6,486 ± 0,076	6,26 ± 0,16	0,52

Taula 28: Mitjana ± desviació estàndard de les execucions afegint la paral·lelització de *feed_input()*.

En afegir aquesta funció s'obté una millora d'entre 1 i 2 segons al llarg de totes les execucions, i fins i tot, la variabilitat dels temps ha disminuït. Continuem afegint paral·lelitzacions i agregem la funció *init_deltas()*.

Fils	Temps (s)	Speedup	Eficiència
1	40,618 ± 0,726	1,00 ± 0,03	1,00
2	20,683 ± 0,122	1,96 ± 0,04	0,98
3	14,286 ± 0,055	2,84 ± 0,05	0,95
4	11,095 ± 0,036	3,66 ± 0,07	0,92
5	9,296 ± 0,044	4,37 ± 0,08	0,87
6	8,316 ± 0,099	4,88 ± 0,11	0,81
7	7,461 ± 0,083	5,44 ± 0,11	0,78
8	6,971 ± 0,139	5,83 ± 0,16	0,73
9	6,613 ± 0,072	6,14 ± 0,13	0,68
10	6,365 ± 0,088	6,38 ± 0,14	0,64
11	6,346 ± 0,138	6,40 ± 0,18	0,58
12	6,487 ± 0,134	6,26 ± 0,17	0,52

Taula 29: Mitjana ± desviació estàndard de les execucions afegint la paral·lelització de *init_deltas()*.

S'ha continuat millorant el temps general, potser de manera no tant notòria en alguns dels casos, però segueix sent una addició positiva, per tant mantenim la funció paral·lelitzada i afegim la següent, *compute_loss()*.

Fils	Temps (s)	Speedup	Eficiència
1	40,809 ± 0,757	1,00 ± 0,03	1,00
2	20,784 ± 0,055	1,96 ± 0,04	0,98
3	14,366 ± 0,052	2,84 ± 0,05	0,95
4	11,158 ± 0,045	3,66 ± 0,07	0,91
5	9,353 ± 0,044	4,36 ± 0,08	0,87
6	8,254 ± 0,065	4,94 ± 0,10	0,82
7	7,424 ± 0,041	5,50 ± 0,11	0,79
8	6,910 ± 0,043	5,91 ± 0,12	0,74
9	6,571 ± 0,071	6,21 ± 0,13	0,69
10	6,380 ± 0,030	6,40 ± 0,12	0,64
11	6,259 ± 0,053	6,52 ± 0,13	0,59
12	6,355 ± 0,193	6,42 ± 0,23	0,54

Taula 30: Mitjana ± desviació estàndard de les execucions afegint la paral·lelització de *compute_loss()*.

Aquí el benefici és quasi nul, millorant el temps i el *speedup* unes dècimes en les execucions amb alt nombre de threads. S'estudiarà si, en afegir l'ultima funció que resta per paral·lelitzar, la paral·lelització de la funció *compute_loss()* aporta un benefici o no. De moment es manté ja que no empitjora el programa.

La taula amb la funció *update_weights()* és la següent.

Fils	Temps (s)	Speedup	Eficiència
1	40,820 ± 0,853	1,00 ± 0,03	1,00
2	20,403 ± 0,053	2,00 ± 0,04	1,00
3	14,037 ± 0,115	2,91 ± 0,07	0,97
4	10,716 ± 0,064	3,81 ± 0,08	0,95
5	8,876 ± 0,031	4,60 ± 0,10	0,92
6	7,748 ± 0,060	5,27 ± 0,12	0,88
7	6,947 ± 0,038	5,88 ± 0,13	0,84
8	6,423 ± 0,043	6,36 ± 0,14	0,79
9	6,024 ± 0,056	6,78 ± 0,16	0,75
10	5,824 ± 0,039	7,01 ± 0,15	0,70
11	5,720 ± 0,044	7,14 ± 0,16	0,65
12	5,757 ± 0,102	7,09 ± 0,19	0,59

Taula 31: Mitjana ± desviació estàndard de les execucions afegint la paral·lelització de *update_weights()*.

Amb aquesta última funció paral·lelitzada s'ha vist una millora d'uns $0,5s$ a partir de l'ús de 4 threads, per tant es manté aquesta paral·lelització.

Finalment es comprova si traient la paral·lelització de la funció *compute_loss()* s'obté algun benefici, ja que s'ha vist que no aportava gran millora per si sola.

Fils	Temps (s)	Speedup	Eficiència
1	41,565 ± 2,008	1,00 ± 0,07	1,00
2	20,620 ± 0,907	2,02 ± 0,13	1,01
3	13,878 ± 0,040	3,00 ± 0,14	1,00
4	10,903 ± 0,829	3,81 ± 0,34	0,95
5	9,270 ± 0,900	4,48 ± 0,49	0,90
6	8,092 ± 0,795	5,14 ± 0,56	0,86
7	6,904 ± 0,026	6,02 ± 0,29	0,86
8	6,338 ± 0,033	6,56 ± 0,32	0,82
9	5,963 ± 0,102	6,97 ± 0,36	0,77
10	5,936 ± 0,443	7,00 ± 0,62	0,70
11	5,957 ± 0,501	6,98 ± 0,68	0,63
12	5,690 ± 0,261	7,30 ± 0,49	0,61

Taula 32: Mitjana ± desviació estàndard de les execucions traient la paral·lelització de *compute_loss()*.

Donades aquestes dades, els temps mitjans són similars entre les dues versions i cap de les dues és més ràpida en totes les configuracions. No obstant això, en diverses execucions s'observa una menor desviació estàndard quan es manté la paral·lelització de *compute_loss()*. Per aquest motiu, s'ha decidit conservar-la.

3.4 Comparació amb les dades de referència

Un cop acabat l'ajust de paràmetres de paral·lelització, i amb una versió definitiva de totes les funcions paral·lelitzades, es compara el temps i l'*speedup* de la nostra versió amb les dades de referència.

Fils	**Ref. (s)**	**Propi (s)**	**Dif. (s)**	**Dif. (%)**	**Execs. < ref.**
2	22,543	20,408 ± 0,044	−2,135	−9,47	20/20
4	11,730	10,707 ± 0,036	−1,023	−8,72	20/20
6	8,309	7,745 ± 0,034	−0,564	−6,79	20/20
12	5,264	5,722 ± 0,135	0,458	8,71	0/20

Taula 33: Comparació del temps d'execució propi (mitjana ± desviació estàndard de 20 execucions) amb el temps de referència. Els valors negatius indiquen un temps inferior al de referència.

Fils	**Speedup ref.**	**Speedup propi**	**Eficiència ref.**	**Eficiència pròpia**
2	1,81	2,00 ± 0,00	0,90	1,00
4	3,47	3,80 ± 0,01	0,87	0,95
6	4,90	5,26 ± 0,02	0,82	0,88
12	7,74	7,12 ± 0,17	0,64	0,59

Taula 34: Speedup i eficiència propis i de referència, calculats amb el temps seqüencial de referència (40,738 s).

Com es pot veure a les taules, el codi paral·lelitzat que hem fet assoleix els temps de referència en els casos de $2, 4$ i 6 threads, però es queda enrere en el cas de 12 threads. El mateix passa amb el *speedup* i l'eficiència.

Tot i això, no està gaire lluny dels paràmetres de referència l'execució amb 12 threads, amb una diferència de $0,458 \pm 0,135s$ en el temps, $0,62 \pm 0,17$ d'*speedup* i $0,05$ punts en l'eficiència.

Respecte als encerts, s'han mantingut estables al llarg de tota la pràctica, excepte en els punts comentats a la secció `Baixada d'encerts sense la directiva #pragma omp single (Condició de carrera)`. Les dades d'aquestes últimes execucions han sigut les següents:

- 923 (/946): 1 execució.
- 927 (/946): 5 execucions.
- 928 (/946): 24 execucions.
- 929 (/946): 46 execucions.
- 930 (/946): 4 execucions.

4 Problemes trobats

4.1 Concepte de SIMD

SIMD és un acrònim que significa *Single instruction, multiple data*, i la directiva d'OpenMP **#pragma omp simd** serveix per executar diverses iteracions d'un bucle simultàniament mitjançant instruccions **SIMD** del processador.

L'avantatge de **SIMD** és que, en lloc de gestionar i executar les operacions de suma i producte individualment per a cada iteració, el processador pot treballar amb grups d'elements mitjançant instruccions vectorials. La mida dels grups depèn del processador, concretament de l'arquitectura d'aquest, i de la mida de les dades.

4.2 Baixada d'encerts sense la directiva `#pragma omp single` (Condició de carrera)

En fer la paral·lelització bàsica, ens vam trobar amb una reducció molt gran del nombre d'encerts a causa de les condicions de carrera que es produïen entre els threads en els dobles bucles de la funció. Inicialment, com que havíem paral·lelitzat les altres parts però no havíem afegit explícitament la directiva `#pragma omp for` en aquests bucles, pensàvem que només els executaria un thread.

Finalment, vam observar que, en trobar-se dins d'una regió `#pragma omp parallel`, tots els threads executaven íntegrament aquests bucles, en lloc de repartir-se'n les iteracions. Això provocava que diversos threads realitzessin els mateixos càlculs i modifiquessin simultàniament les mateixes posicions dels vectors `dw` i `dactv`, generant condicions de carrera en les operacions d'acumulació. Com a conseqüència, el càlcul dels gradients podia ser incorrecte, fet que explicava la gran disminució del nombre d'encerts.

Això ens va portar a utilitzar la directiva `#pragma omp single`, que fa que el bucle sigui executat per un únic thread, com si es tractés d'una execució seqüencial. D'aquesta manera, aconseguim evitar les condicions de carrera i la repetició dels càlculs, recuperant el nombre d'encerts correcte. No obstant això, el temps d'execució d'aquesta part no havia de ser necessàriament igual al de la versió seqüencial, a causa de la sobrecàrrega associada a la gestió dels threads i a la sincronització dins de la regió paral·lela.

4.3 Augment del temps d'execució a causa de l'ús de la directiva `#pragma omp atomic`

Una alternativa per evitar les condicions de carrera en paral·lelitzar els dobles bucles de la funció `back_prop()` era utilitzar la directiva `#pragma omp atomic`.

Aquesta permet que diversos threads executin el bucle simultàniament, garantint que les actualitzacions sobre una mateixa posició de memòria es realitzin de manera atòmica. No obstant això, aquesta sincronització pot incrementar considerablement el temps d'execució quan diferents threads intenten modificar repetidament els mateixos elements del vector `dactv`.

En el nostre cas, aquest problema es produïa perquè paral·lelitzàvem el bucle d'índex j, mentre que cada thread recorria els mateixos valors de k, de manera que diversos threads podien intentar acumular simultàniament les seves contribucions sobre una mateixa posició de `dactv[k]`. Aquest fet era especialment rellevant a les *Hidden Layers*, on els dobles bucles realitzen un total de 645.350 iteracions, corresponents a les combinacions de neurones de les capes 3, 2 i 1 amb les seves capes anteriors. D'aquestes, 133.350 requereixen una acumulació sobre `dactv`, cosa que implicava executar el mateix nombre d'operacions atòmiques per cada crida a `back_prop()`. Com que aquesta funció s'executa repetidament durant l'entrenament, la sobrecàrrega associada a aquestes sincronitzacions podia acabar afectant significativament el temps total d'execució.

Per aquest motiu, es va considerar substituir la directiva `#pragma omp atomic` per la clàusula `reduction`. Aquesta alternativa permet que cada thread acumuli les seves contribucions en una còpia privada del vector `dactv`, evitant la necessitat de sincronitzar individualment cada suma. Un cop finalitzat el bucle, OpenMP s'encarrega de combinar les contribucions de les còpies privades sobre el vector original. D'aquesta manera, es busca reduir la sobrecàrrega de sincronització i millorar el rendiment de la paral·lelització, tot i que la reducció també comporta un cost associat a la creació i combinació dels vectors privats.

5 Conclusions

En aquesta pràctica hem treballat en la paral·lelització d'un programa en C que implementa una xarxa neuronal, amb l'objectiu de reduir-ne el temps d'execució mitjançant OpenMP. Per aconseguir-ho, hem analitzat les diferents funcions del programa i hem estudiat quines parts es podien paral·lelitzar, tenint en compte els possibles problemes que podien aparèixer i les diferents alternatives per solucionar-los.

Els resultats han mostrat que no totes les funcions es beneficien de la paral·lelització individualment, especialment quan presenten poca càrrega computacional i la sobrecàrrega de gestió dels threads és comparable al treball realitzat. En canvi, les funcions `forward_prop()` i `back_prop()` han estat especialment rellevants per aconseguir una reducció del temps total d'execució.

Durant el procés d'optimització, s'han identificat diferents problemes relacionats amb les condicions de carrera, les dependències entre iteracions i el cost de les operacions atòmiques. L'ús de reduccions ha permès paral·lelitzar acumulacions compartides que inicialment s'havien executat de manera seqüencial mitjançant la directiva `#pragma omp single`.

Pel que fa al rendiment, la versió final ha obtingut uns temps mitjans de 20,408 s, 10,707 s, 7,745 s i 5,722 s amb 2, 4, 6 i 12 threads, respectivament. En comparació amb els temps proporcionats com a referència, s'han aconseguit millores amb 2, 4 i 6 threads, mentre que amb 12 threads el temps ha estat lleugerament superior.

Finalment, aquesta pràctica ha permès comprovar que una paral·lelització eficient no consisteix únicament a distribuir iteracions entre threads, sinó també a gestionar adequadament les dependències, reduir la sobrecàrrega de sincronització i escollir una distribució del treball apropiada per a cada situació.

6 Ús de IA

En aquesta pràctica s'ha fet ús d'Intel·ligència artificial per diverses tasques, amb el motiu de l'aclariment de conceptes, correcció d'errors tipogràfics i conceptuals i automatització de tasques repetitives.

S'han usat scripts de Python generats per IA que recollien les dades generades en els `slurm-*.out` i generaven les taules i els gràfics presentats en aquest informe, ja que representava una tasca repetitiva amb càlculs de mitjanes i variàncies que no representen cap complicació conceptual.

També s'ha usat la IA per fer recerca de conceptes, com el concepte de **SIMD** explicat anteriorment. La IA ha sigut una de les eines de recerca, s'ha complementat l'informació amb altres fonts, esmentades en la bibliografia.

Referències

[1] Wikipedia contributors, "Single instruction, multiple data," https://en.wikipedia.org/wiki/Single_instruction,_multiple_data, 2026, wikipedia, The Free Encyclopedia.

[2] OpenMP Architecture Review Board, "OpenMP reference guides," https://www.openmp.org/resources/refguides/, 2025.